MODULE 05 · 28 MIN

Testing, Debugging & Self-Review

Untested AI code is a guess. Make Claude review its own work as a stranger's PR.

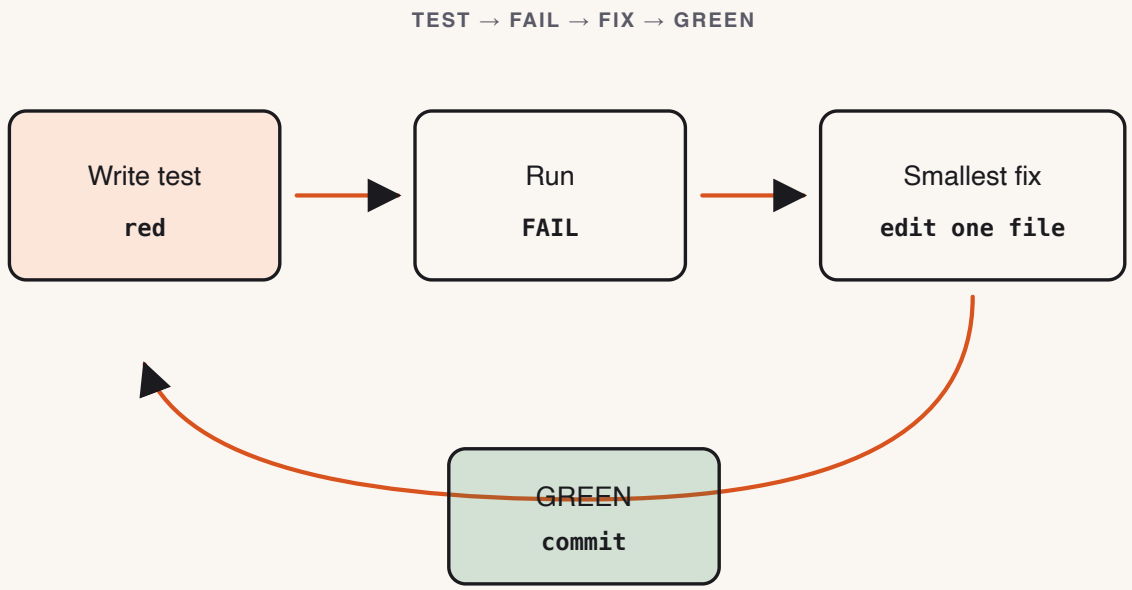
Theory · Test, then self-review (4 min)

Test pyramid for AI code: many cheap unit tests · a few integration tests on the happy path · always cover **error paths**.

> **Self-review prompt**: ask Claude to find bugs *as if reviewing a stranger's PR.* The framing kills sycophancy.

- Off-by-one and **boundary** bugs are Claude's blind spot — always test boundaries.
- Bundled skills cut prompt repetition: `/debug` · `/verify` · `/code-review` · `/loop` · `/batch`.
- You ship a **personal** `code-review-rubric.md` — your blind spots, not the instructor's.

The test-and-review loop



Tests → find the bug → **self-review as a stranger** → fix → re-run until green.

Reference · The self-review prompt

```
Review this code as if it were a stranger's pull request.
List every bug, edge case, and boundary error you can find.
Be specific: file, line, symptom, and the fix. Do not be polite.
```

Test in-process with a temp SQLite DB per test — **no network, no HTTP mocks, never mock the system under test.**

Reference · Common mistakes

- Tests that mock the system under test (useless).
- Self-review without the "stranger's PR" framing (sycophantic output).
- Copying the skill rubric verbatim — your rubric must reflect *your* blind spots.
- Confusing the student rubric with the instructor grading rubric (different files).

Live demo · Plant a bug, catch it (6 min)

1. Open the Module 4 winner; ask for a test suite (pytest + httpx, or vitest + fetch). Run → green.
2. Plant one off-by-one bug live (e.g. a pagination boundary).
3. Paste the **self-review prompt** verbatim:

```
Review this code as if it were a stranger's PR you must approve.
List concrete bugs with file, line, and a minimal fix. Don't say "looks good".
```

4. Claude finds it → fix → re-run. Repeat with a second seeded bug.

Success signal: the self-review names the bug's file, line, and fix — not "looks good".

Your turn · Suite + 2 bugs + your rubric (11 min)

Exercise: `exercises/part-05/README.md`

STEP 1 Write a full suite: create, list, search, get-one, update, delete, 404, 422.

STEP 2 Plant **two** seeded bugs (from the reference), use the self-review prompt to fix them.

STEP 3 Author `code-review-rubric.md` — ≤ 1 page, 5–8 checks, focused on Claude's blind spots.

Deliverables: green suite · `bug-fix-notes.md` (symptom → cause → diagnosis → fix) · personal rubric.

Success signal: tests pass on fixed code; rubric has ≥ 1 check not in `skills/code-review/SKILL.md`.

✓ Done & next (1 min)

Definition of done

- [ ] Test suite runs green on fixed code.
- [ ] `bug-fix-notes.md` documents both bugs (symptom, cause, diagnosis, fix).
- [ ] Personal rubric with ≥ 1 original check.

Next — tested code earns a safe path to main: branches, atomic commits, a real PR.
Module 6 — Git Workflows for Safe AI Dev.